

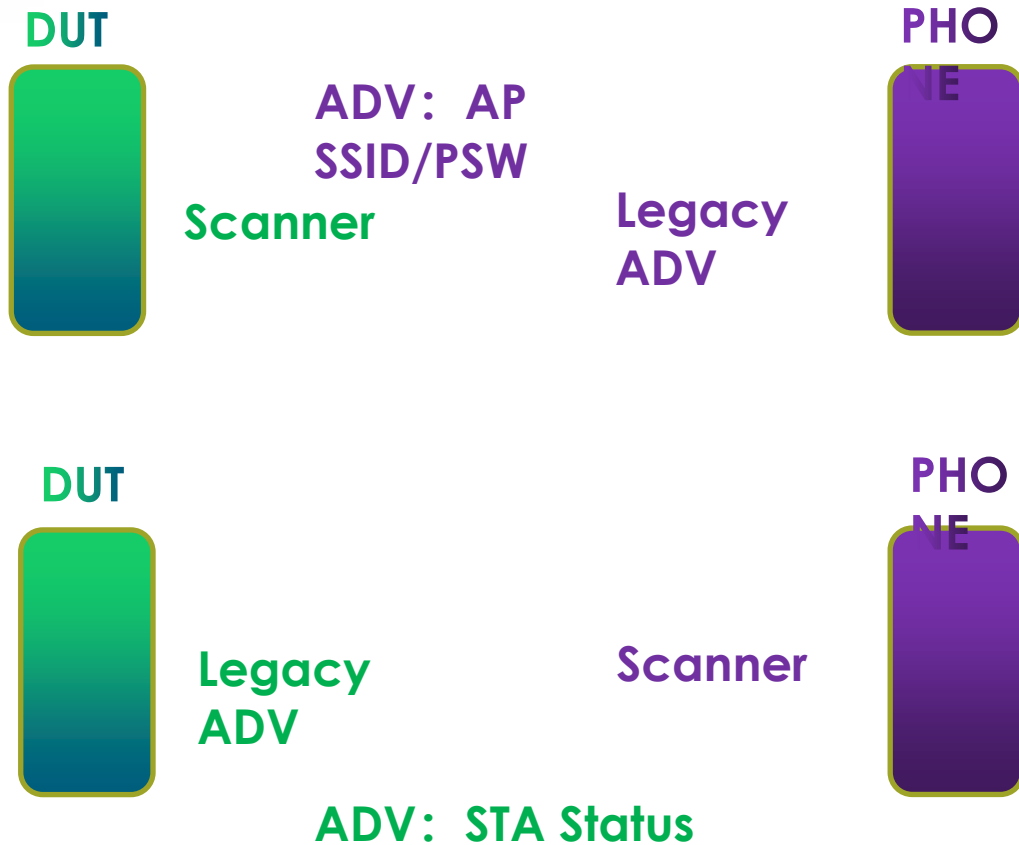
[6030] 支援BLE廣播配網

SW MAC team

需求

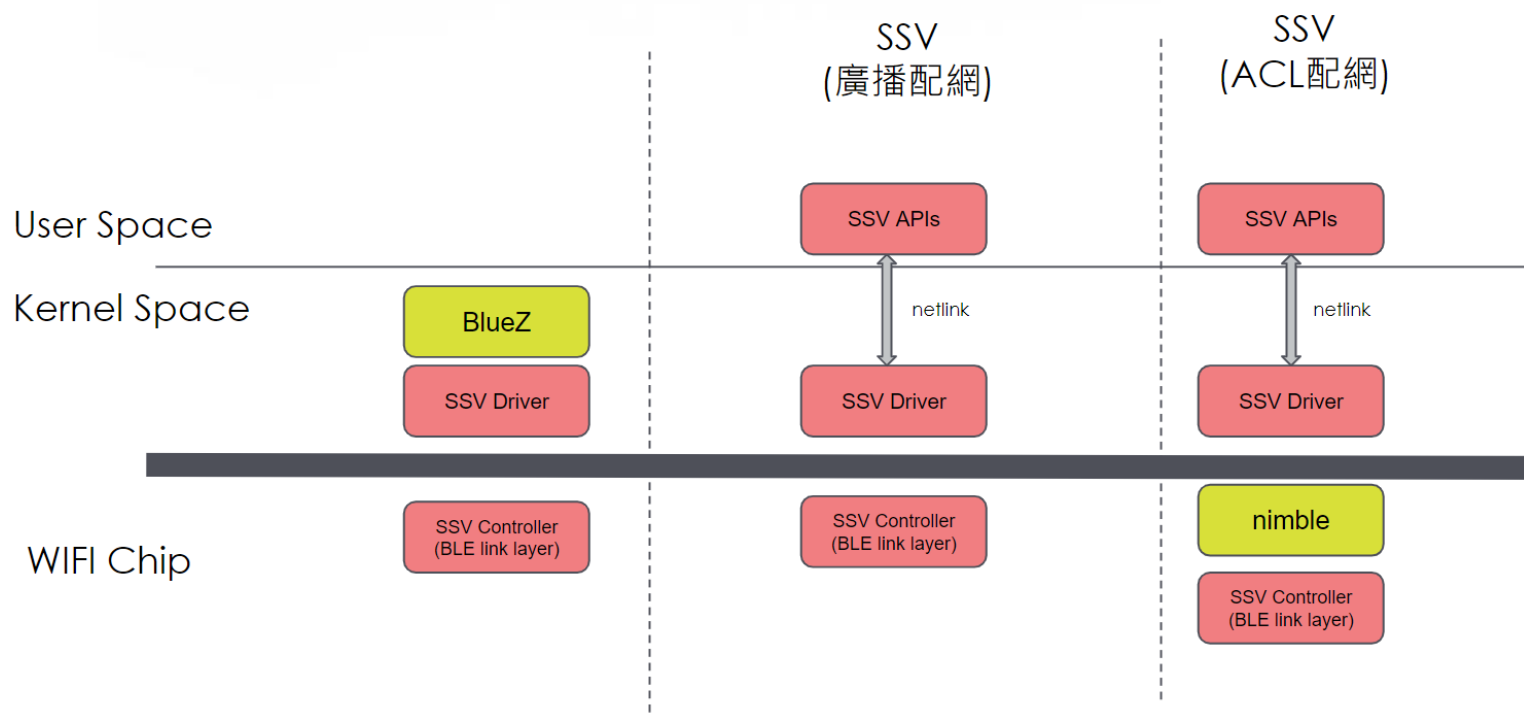
- 前線FAE訪查客戶在BLE配網的需要，整理出兩個小點：
 1. ACL配網: BLE 需要連線，需要BLE stack (GATT/GAP....profile)
 2. 廣播配網: 無須連線，只需要廣播/SCAN。

➤ 配网 (ADV)



目的

- 由於廣播/scan無需BLE stack, 直接對controller (Link Layer)發出HCI cmd命令它工作, 希望藉由ssv6x5x-sw_ble.bin, 但不需要Linux BlueZ協助下, 完成我們的目標。



Host driver啟動BLE收發app data

在<ssvdriver folder>/目錄下

1. 編輯config.mak, 打開compiler flag, (新增CONFIG定義**BLE_USE_LL_ONLY**)

```
## NIMBLE使用((注意与BLE的使用为2选1))
## 使用ssv6x5x-sw_nimble.bin
#ccflags-y += -DCONFIG_NIMBLE

## BLE HCI usage (注意与NIMBLE的使用为2选1)
## 使用ssv6x5x-sw_ble.bin
CONFIG_BLE ?= y
ccflags-$(CONFIG_BLE) += -DCONFIG_BLE
## BLE使用application收發data (需同時打開CONFIG_BLE與
CONFIG_BLE_HCI_OVER_HWIF方能使用)
CONFIG_BLE_USE_LL_ONLY ?= y
ccflags-$(CONFIG_BLE_USE_LL_ONLY) += -DBLE_USE_LL_ONLY
```

2. 編譯driver

```
# make
```

3. 修改ssv6x5x-wifi.cfg使用BLE firmware(ssv6x5x-sw_ble.bin)

```
#####
# Firmware setting
#
# Priority.1 insmod parameter "cfgfirmwarepath" (insmod的時候帶入)
# Priority.2 firmware_path (在下面設定)
# Priority.3 default firmware (前兩個都沒設定會load firmware header)
#
#####
#firmware_path = /home/xxx/xxx/image/
firmware_path = /home/ssv/work/svn/host/FMAC/branch/L.FWB.23Q1.0000.00/image/
#####
# Firmware name
#
# 若有設定firmware path的話才有用, 內部預設firmware name使用"ssv6x5x-sw.bin"
#
#####
#firmware_name = ssv6x5x-sw.bin
firmware_name = ssv6x5x-sw_ble.bin
```

4. 載入driver

```
# ./load.sh
```

BLE app收發BLE LL的data

在<ssvdriver folder>/tools/ble/ble_app/目錄下

1. 編輯Makefile, 打開compiler flag, (新增**BLE_USE_LL_ONLY**)

```
CC = gcc
CFLAGS = -c -Wall -fshort-enums
CFLAGS += -DBLE_USE_LL_ONLY
LDFLAGS =
```

2. 編譯ble_app
3. 執行ble_app, 並CLI command “**init=1**”

```
# ./ble_app

ble> init=1
bleinit=1
ble: HCI Reset
ble: set event mask
ble: set le event mask

ble> BLE RX: Command Complete. len=7
BLE RX: Command Complete. len=7
BLE RX: Command Complete. len=7
```

CLI指令範例

1. init

```
ble> init=1
bleinit=1
ble: HCI Reset
ble: set event mask
ble: set le event mask

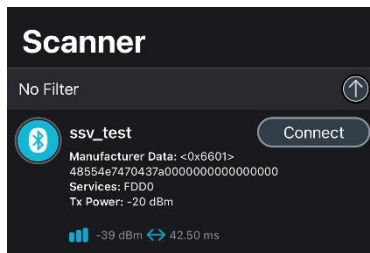
ble> BLE RX: Command Complete. len=7
BLE RX: Command Complete. len=7
BLE RX: Command Complete. len=7
```

2. advert

```
ble> advert=1,ssv_test
bleADV=1,ssv_test
ble: set advertising data
ble: set scan response data
ble: set advertising parameters
ble: set advertising enable

ble> BLE RX: Command Complete. len=7
BLE RX: Command Complete. len=7
BLE RX: Command Complete. len=7
BLE RX: Command Complete. len=7
```

(手機應用程式nRF Connect的scan結果)



3. scanparam

```
ble> scanparams=1,0,0,320,48,1
blestartscan=1,0,0,320,48,1
ble: set scan parameter

ble> BLE RX: Command Complete. len=7
```

4. scan

```
ble> scan=1
blestartscan=1
ble: scan enable

ble> BLE RX: Command Complete. len=7
BLE RX: LE Advertising Report. len=43
BLE RX: LE Advertising Report. len=25
BLE RX: LE Advertising Report. len=34
BLE RX: LE Advertising Report. len=32
BLE RX: LE Advertising Report. len=15
BLE RX: LE Advertising Report. len=30
BLE RX: LE Advertising Report. len=30
```

實作項目

1. BLE APP TX:

1. init:
 - 1) 通知BLE DRV做init
 - 2) 發"HCI Reset", "set event mask", "set le event mask"命令
2. advert:
 - 1) 發"set advertising parameters", "set advertising data", "set scan response data", and "set advertising enable"命令
3. scanparams
 - 1) 發"set scan parameters"命令
4. scan
 - 1) 發"scan enable"命令

2. BLE APP RX

1. RX packet handler callback
=> 提供callback註冊, 當有RX packet時呼叫callback處理

3. BLE DRV

1. init: 做BLE init, 但不向BlueZ註冊device
2. TX: 透過netlink打來的TX packet送往firmware
3. RX: RX packet透過netlink送往APP

BLE LL API

```
typedef void (*ssv_ble_ll_rx_pkt_cb_t)(uint8_t *, uint32_t);    ///< BLE LL RX packet callback type

/// Ble scan parameters
typedef struct {
    uint8_t          scan_type;          /*!< Scan type */
    uint8_t          own_addr_type;      /*!< Owner address type */
    uint8_t          scan_filter_policy; /*!< Scan filter policy */
    uint16_t         scan_interval;      /*!< Scan interval. This is defined as the time interval from
                                         when the Controller started its last LE scan until it begins the subsequent LE scan.
                                         Range: 0x0004 to 0x4000 Default: 0x0010 (10 ms)
                                         Time = N * 0.625 msec
                                         Time Range: 2.5 msec to 10.24 seconds*/
    uint16_t         scan_window;        /*!< Scan window. The duration of the LE scan. LE_Scan_Window
                                         shall be less than or equal to LE_Scan_Interval
                                         Range: 0x0004 to 0x4000 Default: 0x0010 (10 ms)
                                         Time = N * 0.625 msec
                                         Time Range: 2.5 msec to 10.240 msec */
    uint8_t          scan_duplicate;     /*!< The Scan_Duplicates parameter controls whether the Link Layer should filter out
                                         duplicate advertising reports (BLE_SCAN_DUPLICATE_ENABLE) to the Host, or if the Link Layer should generate
                                         advertising reports for each packet received */
} ssv_ble_ll_scan_params_t;

/// Advertising parameters
typedef struct {
    uint16_t         adv_int_min;        /*!< Minimum advertising interval for
                                         undirected and low duty cycle directed advertising.
                                         Range: 0x0020 to 0x4000 Default: N = 0x0800 (1.28 second)
                                         Time = N * 0.625 msec Time Range: 20 ms to 10.24 sec */
    uint16_t         adv_int_max;        /*!< Maximum advertising interval for
                                         undirected and low duty cycle directed advertising.
                                         Range: 0x0020 to 0x4000 Default: N = 0x0800 (1.28 second)
                                         Time = N * 0.625 msec Time Range: 20 ms to 10.24 sec Advertising max interval */
    uint8_t          adv_type;           /*!< Advertising type */
    uint8_t          own_addr_type;      /*!< Owner bluetooth device address type */
    uint8_t          peer_addr[6];       /*!< Peer device bluetooth device address */
    uint8_t          peer_addr_type;     /*!< Peer device bluetooth device address type, only support public address type and random address type */
    uint8_t          channel_map;        /*!< Advertising channel map */
    uint8_t          adv_filter_policy; /*!< Advertising filter policy */
} ssv_ble_ll_adv_params_t;

int32_t ssv_ble_ll_set_rx_pkt_cb(ssv_ble_ll_rx_pkt_cb_t rx_pkt_cb);
int32_t ssv_ble_ll_rx_pkt_handler(uint8_t *rx_packet, uint32_t rx_len);
int32_t ssv_ble_ll_init(void);
int32_t ssv_ble_ll_deinit(void);
int32_t ssv_ble_ll_set_advertising_data(uint8_t *raw_data, uint32_t raw_data_len);
int32_t ssv_ble_ll_set_scan_rsp_data(uint8_t *raw_data, uint32_t raw_data_len);
int32_t ssv_ble_ll_start_advertising(ssv_ble_ll_adv_params_t *adv_params);
int32_t ssv_ble_ll_stop_advertising(void);
int32_t ssv_ble_ll_set_scan_params(ssv_ble_ll_scan_params_t *scan_params);
int32_t ssv_ble_ll_start_scanning(uint32_t duration);
int32_t ssv_ble_ll_stop_scanning(void);
```

注意事項

1. 因著這次ssv_ctl_common.h有更新, 為了避免出錯, 請**特別注意驅動與應用程式都需同時編譯更新**, 因為兩者都有使用到此header file

Thank you